

AGENCYOS

Phase 1 Agent Interaction & Handoff Specification

Lead-to-Close Workforce | Triggers • Inputs • Outputs • States • Events • Dependencies • Evidence

PDF 2: This document converts the already-created Phase 1 flow and five agent responsibility documents into a single implementation-level interaction contract. It does not replace those documents; it defines how the five agents communicate and hand work between them.

Source basis: Uploaded AI Agent Architecture, Orchestrator/Routing, CRM/Sales, Automation/Events/Jobs, API/Integration, Memory/Context and Approval/Policy specifications. The source requires typed tasks/events, minimum relevant context, backend-controlled state, evidence-based completion and safe retries. ■filecite■turn10file4■ ■filecite■turn10file5■

1. Master Interaction Principle

- Agents communicate through typed tasks/events, not conversational memory alone.
- Every handoff identifies sender and destination capability/agent.
- Project/client/organization scope is explicit where applicable.
- Task type and current workflow state are explicit.
- Approved scope and artifact/version references are explicit.
- Required output and acceptance criteria are explicit.
- Policy/approval references are carried when relevant.
- Evidence and blockers are carried.
- Retry count and prior failure summary are carried when relevant.
- Database/workflow engine is authoritative for state.
- Agent-generated content is untrusted until validated.
- COMPLETE requires configured DoD plus evidence.

2. Phase 1 Workforce

Agent	Owns	Does not own
Sales	Client conversation, qualification, discovery, trust, commercial conversation within policy	Routing, scheduling execution, policy override
Scheduler	Availability, slots, booking, reminders, meeting execution	Sales qualification, pricing
Quotation Master	Quotation construction, versions, commercial application within policy	Client sales, approval authority
Coordination	Workflow handoff, dependencies, task/result tracking, blockers	Provider/model selection, domain execution
Orchestrator/Router	Agent/provider/model routing, execution control, retry/fallback	Business policy override, domain ownership

3. Canonical Handoff Envelope

- task_id
- correlation_id
- organization_id / tenant scope
- client_id
- lead_id
- project_id where applicable
- source_agent
- destination_agent/capability
- task_type
- current_workflow_state
- validated intent / requested action
- approved_scope_reference
- requirement_version
- artifact/version IDs

- relevant context references
- required_output_schema
- acceptance_criteria
- deadline / priority
- policy_decision_reference
- approval_reference
- evidence_links
- known_blockers
- retry_count
- previous_failure_summary
- idempotency_key for side effects
- security/data-classification metadata

4. Canonical Result Envelope

- task_id and correlation_id match
- result_status
- returning agent
- structured result payload
- artifact/version references
- evidence references
- validation status
- warnings
- blocker if incomplete
- error/failure classification
- attempt number
- provider/model metadata where available
- usage/cost metadata where available
- next-action recommendation
- completion/DoD evidence

5. State Rules

Recommended task lifecycle: CREATED → VALIDATING → READY → DISPATCHED → ACKNOWLEDGED → IN_PROGRESS → WAITING_FOR_RESULT → RESULT_RECEIVED → VALIDATING_RESULT → ACCEPTED → HANDOFF_READY → HANDED_OFF → VERIFIED → CLOSED. Exception states: BLOCKED, RETRYING, FAILED, EXPIRED, CANCELLED, ESCALATED.

- Backend/workflow engine owns authoritative transitions.
- Invalid transitions are rejected.
- Every transition records actor, timestamp and reason.
- Result arrival is not automatically acceptance.
- Completion requires configured evidence.
- Paused workflows remain resumable.
- Failed steps can retry or escalate.
- Sensitive conditions are evaluated immediately before execution.

6. End-to-End Phase 1 Interaction Map

LEAD EVENT → COORDINATION → ORCHESTRATOR → SALES → COORDINATION → [SCHEDULER WHEN NEEDED] → SALES → REQUIREMENTS CONFIRMED → COORDINATION → ORCHESTRATOR → QUOTATION MASTER → POLICY/ADMIN GATE → SALES DELIVERY → NEGOTIATION LOOP → EXACT ACCEPTANCE → PAYMENT/EXCEPTION GATE → WON → HANDOFF.

The route branches by client intent and policy. A meeting is optional; quotation revision can repeat; approval can be automatic or Admin-gated. Every branch must become durable state.

7. Interaction 01 — Lead Created → Sales

- Trigger is verified CRM lead-created event.
- Coordination validates lead existence.
- Duplicate/open-lead checks complete.
- Sales owner is known/configured.
- One idempotent Sales task is created.
- Orchestrator validates Sales capability/permissions.
- Orchestrator selects approved model/provider.
- Sales receives minimum relevant context.
- Sales returns structured result/progress.
- Orchestrator validates result.
- Coordination records result and next action.

Example: Facebook lead L-1001 arrives. Coordination creates T-1001. Orchestrator routes T-1001 to Sales. Sales returns qualification and next action. Coordination stores it and determines the next valid step.

8. Interaction 02 — Sales → Meeting Intent → Scheduler

- Sales detects genuine call/meeting request.
- Scheduling intent is structured.
- Coordination creates Scheduler task.
- Task includes identity, purpose, preferred time/range and timezone when known.
- Orchestrator validates Scheduler capability.
- Orchestrator checks required tools/permissions.
- Scheduler checks availability.
- Scheduler proposes or books slot.
- Result contains booking reference/state.
- Orchestrator validates result.
- Coordination records exact slot/timezone.
- Sales receives confirmation.
- Duplicate booking attempts are idempotently prevented.
- No agent invents availability.

9. Interaction 03 — Scheduler Lifecycle

- Create task
- Validate identity/constraints
- Check availability
- Propose alternatives if needed
- Book confirmed slot
- Store provider/booking reference
- Trigger configured reminders
- Process reschedule
- Process cancellation
- Process no-show
- Record meeting completion
- Ingest recording/notes/transcript references where configured
- Trigger meeting-analysis task where configured
- Validate analysis output
- Return evidence-backed outcome to Sales
- Retry safely or escalate on failure

10. Interaction 04 — Meeting Result → Sales

- Meeting completion correlates to correct lead/client.
- Recording/notes/transcript references are validated.
- Analysis task is created if configured.
- Analysis separates confirmed decisions from inference.
- Important facts carry provenance.
- Outcome includes evidenced requirements/objections/next action.
- Coordination sends structured outcome to Sales.
- Sales continues using returned context.
- Private internal analysis is not exposed to client.

11. Interaction 05 — Requirements Confirmed → Quotation Master

- Sales has structured requirements.
- Confirmed facts are separated from assumptions.
- Mandatory quote inputs are complete or explicitly pending.
- Requirement/scope version exists.
- Coordination checks quote readiness.
- Quote task is created.
- Orchestrator validates Quotation capability.
- Exact requirement/scope version is passed.
- Approved commercial configuration references are passed.
- Quotation Master returns structured quote object/version.
- Orchestrator validates result.
- Coordination records quote version.

Typical requirement context: service/project type, target users, platforms, core features, integrations, design references, timeline, budget where appropriate, urgency, decision-maker, existing assets, special requirements, language and trust concerns. ■filecite■turn10file13■

12. Interaction 06 — Quotation → Policy/Admin Approval

- Quote ID/version exists.
- Commercial validation is complete.
- Policy determines approval requirement.
- Auto-approved result is recorded when configured.
- Admin approval request is created when required.
- Approval references exact quote version.
- Approval reason/evidence/policy version are stored.
- Approval state is separate from quote state.
- Changes requested return to quote owner.
- Material quote change creates new version.
- Stale approval is invalidated/re-evaluated.
- Rejected quote cannot proceed to delivery.

13. Interaction 07 — Approved Quote → Client

- Exact approved version identified.
- Delivery channel permitted.
- Communication policy/template checks pass.
- Delivery task/event created.
- Exact quote artifact attached.
- Message/document ID recorded.
- Delivery/read status tracked where supported.
- Sales receives response events.

- Delivery/view does not equal acceptance.
- High-risk outbound action remains approval-gated where configured.

The API specification defines WhatsApp outbound as trigger → communication policy → template/content check → recipient check → provider API → message ID → delivery tracking → audit. ■filecite■turn10file18■

14. Interaction 08 — Objection → Negotiation → New Version

- Sales identifies objection.
- Objection type/concern structured.
- Coordination records next action.
- Negotiation round recorded.
- Requested change classified as price/discount/scope/payment/timeline/other.
- Policy limits checked.
- Quotation revision dependency identified.
- Admin approval dependency identified.
- Orchestrator routes execution task.
- Quotation Master creates new version for material change.
- Historical versions remain immutable.
- Negotiation result links to exact version.
- Configured limits stop autonomous negotiation.
- Unclear authority escalates.

15. Interaction 09 — Exact Acceptance

- Sales receives acceptance signal.
- Coordination identifies exact quote version.
- Identity/evidence validated.
- Terms unambiguous.
- Ambiguous 'okay' with multiple versions triggers clarification.
- Timestamp stored.
- Accepted version stored.
- Accepted payment terms stored.
- Acceptance evidence linked.
- No acceptance inferred from delivery/view/payment discussion/unrelated chat.
- Backend workflow changes quote/lead state only after valid evidence.

16. Interaction 10 — Payment/Exception → WON

- Accepted quote/version passed to configured Finance/payment workflow.
- Acceptance and payment remain separate states.
- Payment proof/reference recorded.
- Authorized verification occurs where required.
- Special-payment/no-advance exception preserves approver/reference.
- Conversion conditions evaluated immediately before WON.
- Blocking approval prevents WON.
- Verified gates allow backend-controlled WON.
- Existing client identity reused when reliable match exists.
- Structured onboarding/project handoff event created.

The onboarding source defines WON → client validation → onboarding → project creation → team assignment → WhatsApp group → scope/dates → kickoff → active project, with accepted quotation/version and required payment condition or approved exception.

■filecite■turn10file12■

17. Coordination ↔ Orchestrator Contract

- Coordination records workflow next action.
- Coordination creates operational handoff task.
- Orchestrator receives executable AI task.
- Orchestrator selects specialist.
- Orchestrator selects model/provider.
- Orchestrator enforces routing constraints.
- Orchestrator executes and validates structured result.
- Coordination consumes result and updates workflow progress.
- Neither bypasses Policy/Admin.
- Routing and workflow handoff audits remain distinguishable.

18. Orchestrator ↔ Specialist Contract

- Required capability explicit.
- Specialist eligible.
- Model/provider supports requirements.
- Context authorized/minimized.
- Tool permissions valid.
- Output schema known.
- Acceptance criteria known.
- Execution traceable.
- Result schema validated.
- Fallback preserves task/correlation identity.

19. Specialist ↔ Specialist Rule

- Use structured tasks/events.
- Recipient validates capability and current state.
- Sender cannot grant permissions it does not own.
- Event payload is data, not authorization.
- High-risk action still requires policy/approval.
- Agent loops are detected/limited.
- Direct specialist-to-specialist AI execution is allowed only if explicitly defined by architecture; otherwise Orchestrator remains routing layer.

20. Event Contract

- Unique event ID
- Event type
- Sender agent
- Recipient capability/agent
- Task ID
- Correlation ID
- Project/client/organization scope where applicable
- Authoritative record references
- Timestamp
- Idempotency key where needed
- Processing status
- Duplicate delivery is harmless
- Event cannot grant permissions

21. Dependency & Version Rules

- Dependent task references prerequisites.

- Prerequisite must pass before dependent execution.
- Upstream version change triggers downstream readiness check.
- Obsolete quote/version work is not reused.
- Safe independent work may run in parallel.
- Conflicting mutable work is serialized.
- Circular dependencies detected.
- Stalled dependencies escalated.
- Paused workflow resumes from durable state.

22. Retry / Fallback Contract

- Classify failure first.
- Retry only when safe.
- Use retry budget.
- Use backoff.
- Preserve correlation.
- Use idempotency for side effects.
- Carry previous failure summary.
- Fallback only to eligible equivalent route.
- Never blindly retry payment/message/destructive operations.
- Escalate after exhaustion.
- Failed task cannot become complete without recovery evidence.

The Orchestrator source explicitly specifies timeout retry, transient provider retry/alternate provider, rate-limit fallback, schema-failure correction/alternate model, safe tool retry and escalation after repeated failure. ■filecite■turn10file0■

23. Context & Memory Contract

- Use client memory only when relevant.
- Lead memory supports sales conversation/qualification/objections.
- Project memory applies once project context exists.
- Prefer artifact references over copying large documents.
- Important facts carry provenance.
- AI assumptions are not verified facts without evidence.
- Memory access is permissioned.
- Conflicts trigger validation, not guessing.
- Context size is minimized.
- Stale memory can be detected.

The Memory specification defines memory as events + structured state + summaries + semantic index + artifact references + handoffs + audit, with relevant-context and provenance requirements. ■filecite■turn10file17■

24. Security Contract

- Tenant isolation enforced.
- Least privilege.
- Provider/model permissions separate from agent permissions.
- Secrets remain server-side.
- Sensitive context minimized before provider routing.
- Provider responses are untrusted content.
- Client-facing agents cannot access internal credentials/private strategy.
- High-risk tool calls require backend authorization and configured approval.
- Cross-client context blocked.
- Critical actions audited.

25. Completion Contract

- Input validation passed.
- Policy check passed.
- Task executed.
- Output/schema validation passed.
- Business-rule validation passed where applicable.
- Required evidence captured.
- Required approval exists.
- Acceptance criteria satisfied.
- State transition valid.
- Downstream handoff package complete.
- Only then may task be COMPLETE.

The source architecture explicitly states that the Orchestrator's completion condition is a downstream task accepted or correctly escalated, and that handoff occurs only after acceptance criteria are satisfied. ■filecite■turn10file14■

26. Implementation Build Order

26. Implementation Build Order

- Contract Layer: task schema; event schema; result schema; handoff schema; error schema; version/evidence references.
- Workflow Layer: Phase 1 state map; trigger matrix; prerequisites; completion gates; blocked/escalated states.
- Coordination Layer: Lead→Sales; Sales→Scheduler; Scheduler→Sales; Sales→Quotation; Quotation→Approval; negotiation/version; acceptance→close; WON handoff.
- Orchestrator Layer: capability registry; routing policy; model/provider selection; execution envelope; result validation; retry; fallback; provider health.
- Reliability Layer: idempotency; concurrency protection; durable events/jobs; timeouts; dead-letter/escalation; resume/cancel.
- Security Layer: tenant isolation; permissions; data classification; secret isolation; audit.
- Observability Layer: correlation tracing; task logs; routing logs; failure metrics; handoff latency; escalation metrics.

27. Test Matrix

- Lead event creates exactly one Sales task.
- Duplicate lead webhook does not duplicate task.
- Sales receives minimum authorized context.
- Sales result is schema-valid.
- Meeting request creates one Scheduler task.
- Scheduler result returns exact slot/timezone.
- Booking retry cannot double-book.
- Meeting completion maps to the correct lead.
- Meeting analysis preserves provenance.
- Requirement version is mandatory for quote handoff.
- Quotation routes to Quotation Master.
- Out-of-policy quote cannot bypass approval.
- Approval references exact quote version.
- Material quote revision creates a new version.
- Old approval cannot silently authorize a new version.
- Quote delivery does not set acceptance.
- Ambiguous acceptance triggers clarification.
- Payment verification is separate from acceptance.
- WON cannot occur without configured gates.
- All handoffs appear in audit.
- Unauthorized cross-tenant context is blocked.

- Provider failure invokes safe fallback.
- Unsafe side effect is not blindly retried.
- Deadlock is detected.
- Blocked task is visible and recoverable.
- Every completion has evidence.

28. End-to-End Reference Scenario

Lead L-1001 → Coordination T-1 → Orchestrator → Sales → meeting request → Coordination T-2 → Orchestrator → Scheduler → booking B-1 → Sales → meeting completed → analysis M-1 → requirements R-1 → Coordination T-3 → Orchestrator → Quotation Master → Quote V1 → Policy/Admin approval → delivery → objection → negotiation → Quote V2 → approval → delivery → exact V2 acceptance → payment/exception gate → WON → structured onboarding handoff.

Every ID is correlated.

Every handoff has sender and recipient.

Every result has evidence.

Every material version change is preserved.

Every approval references exact version.

Every failure is recoverable/auditable.

No agent crosses its authority boundary.

Final state is produced by backend/workflow rules.

29. Definition of Done for PDF 2 Implementation

- Canonical task/event/result contracts implemented.
- All five Phase 1 agents have explicit interaction boundaries.
- All Lead-to-Close triggers are mapped.
- All required handoff inputs and outputs are defined.
- State transitions are backend-controlled.
- Dependencies are explicit.
- Retry and fallback are bounded.
- Idempotency protects side effects.
- Policy and approval gates are explicit.
- Exact quote-version acceptance is enforced.
- Security and tenant isolation are enforced.
- Audit/correlation tracing works.
- Failure/escalation states work.
- End-to-end scenario passes.
- QA evidence is stored.
- No unresolved interaction ambiguity remains.

30. Final Interaction Contract

FLOW: Event → Validate → Identify capability → Coordination task → Orchestrator routing → Specialist execution → Result validation → Evidence → Coordination state update → Next task / wait / retry / escalate.

Hard rules: workflow state is authoritative; agent messages are not authority; provider responses are untrusted; approval is an explicit state; versions are exact; completion requires evidence; failures remain failures until safely recovered or escalated.